Developing a Recipe App Using Gen AI-Assisted Code Generation

Project	Course-end Project 2 — "Recipe Book" React app
Submitted by	Nikhil Kommineni
Date	31 October 2026
Tech stack	React 19, React Router 7, Vite, oxlint, Playwright

1. Overview

Recipe Book is a simple, easy-to-use recipe app built with React. People often find recipe sites overwhelming, so this app focuses on the essentials: a clear list of recipes, a detailed card for each recipe, fast search, and a form to add new recipes.

The app is populated with 12 generated sample recipes from 8 cuisines. All seven project tasks are implemented, and the app is checked by 42 automated end-to-end tests, all of which pass.

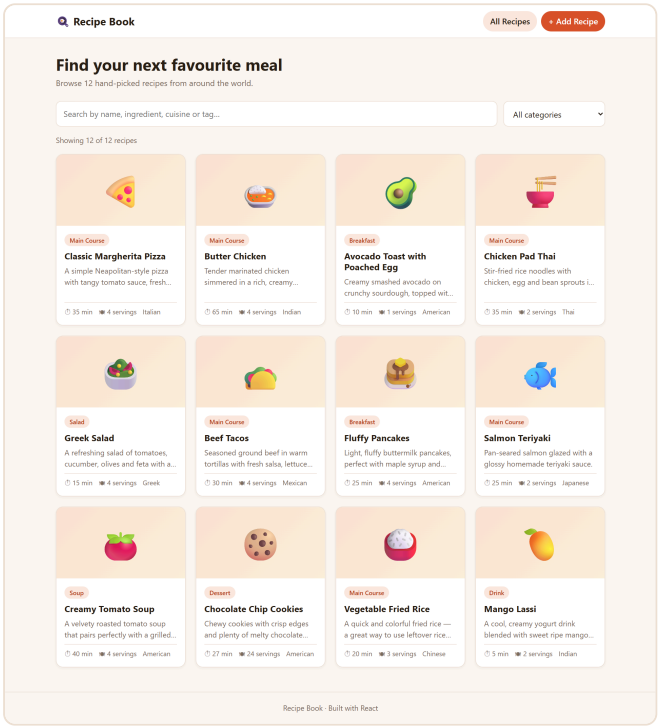


Figure 1 — The recipe list page.

2. Features

- **Recipe list** — every recipe as a card showing category, title, description, total time, servings and cuisine.
- **Recipe detail** — a detail card with prep, cook and total time, servings, difficulty, ingredients, numbered steps and tags.
- **Search** — searches titles, descriptions, cuisines, categories, tags and ingredients, with a category filter. Clicking a tag on a recipe searches for that tag.

- **Add recipes** — a form with validation. New recipes are saved in the browser (localStorage) and survive a refresh.
- **Responsive** — works on desktop and mobile screens.

3. Implementation by task

Task 1 — Set up the project

The project was created with `npx` using Vite's React template, and React Router was added for page navigation:

```
npx create-vite@latest recipe-app --template react
cd recipe-app
npm install
npm install react-router-dom
npm run dev
```

Vite's boilerplate (logos, demo counter) was removed. The app has three routes:

Path	Page	Component
`/`	All recipes with search	`RecipeList`
`/recipes/:id`	Recipe detail	`RecipeDetail`
`/add`	Add a new recipe	`AddRecipeForm`
any other path	Not found	`NotFound`

Task 2 — Create a basic UI

`Header.jsx` provides the logo and navigation ("All Recipes", "+ Add Recipe"), with the current page highlighted. `App.jsx` wraps the pages in a centred container with a footer. Styling lives in `index.css` (theme colours as CSS variables) and `App.css` (layout, cards and forms), with breakpoints at 700px and 480px for small screens.

Task 3 — Create sample data

`src/data/recipes.js` exports 12 generated recipes covering Italian, Indian, American, Thai, Greek, Mexican, Japanese and Chinese cuisine, across six categories (Breakfast, Main Course, Salad, Soup, Dessert, Drink). Each recipe uses the same shape:

```
{
  id: 1,
  title: 'Classic Margherita Pizza',
  emoji: '🍕',
  cuisine: 'Italian',
  category: 'Main Course',
  difficulty: 'Medium',
  prepTime: 20, cookTime: 15, servings: 4,
  description: 'A simple Neapolitan-style pizza with tangy tomato sauce...',
  ingredients: ['500 g pizza dough', '200 g crushed San Marzano tomatoes', ...],
  instructions: ['Preheat the oven to its highest setting...', ...],
```

```
  tags: ['vegetarian', 'baked', 'cheese'],
}
```

Task 4 — Display recipes in the UI

`RecipeList.jsx` renders a grid of `RecipeCard` components. Each card is a link to its detail page and shows the total time, calculated as `prepTime + cookTime`. The grid uses `repeat(auto-fill, minmax(250px, 1fr))`, so the number of columns adjusts to the screen width.

Task 5 — Add more recipes

`App.jsx` combines the sample recipes with recipes the user adds. User recipes are saved to localStorage, so they persist between visits:

```
const [customRecipes, setCustomRecipes] = useState(loadCustomRecipes)

useEffect(() => {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(customRecipes))
}, [customRecipes])

const addRecipe = (recipe) => {
  const nextId = Math.max(...recipes.map((r) => r.id)) + 1
  const newRecipe = { ...recipe, id: nextId }
  setCustomRecipes((prev) => [...prev, newRecipe])
  return newRecipe
}
```

`AddRecipeForm.jsx` collects every field. Ingredients and instructions are entered one per line, and tags are comma separated. The form checks that the name, cuisine, description, at least one ingredient, at least one step, and servings of 1 or more are provided. Each error appears under its field and disappears as soon as that field is edited. After saving, the app opens the new recipe's detail page. If the saved data is ever corrupted, the app ignores it instead of crashing.

Task 6 — Recipe detail card

`RecipeDetail.jsx` reads the recipe id from the URL with `useParams` and shows a card with category and cuisine badges, the description, a stats row (prep, cook, total time, servings, difficulty), an ingredient checklist, numbered instructions, and tags. Each tag links to a search for that tag. The "Back to recipes" button returns to the previous page with the search kept, or to the list if the page was opened directly. Unknown ids show a "Not found" message.

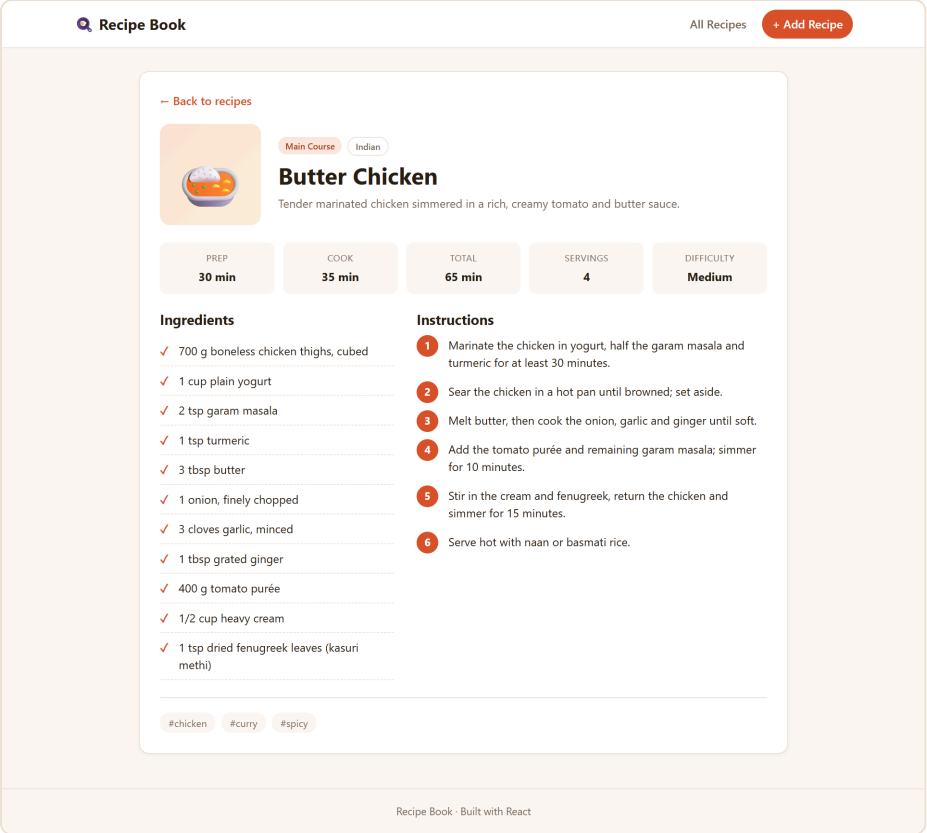


Figure 2 — The recipe detail card.

Task 7 — Search

`SearchBar.jsx` provides a search box and a category dropdown. A recipe matches when every word of the query appears somewhere in its text, ignoring capitals:

```
function matchesQuery(recipe, query) {
  const haystack = [recipe.title, recipe.description, recipe.cuisine,
    recipe.category, ...recipe.tags, ...recipe.ingredients].join(' ').toLowerCase()

  return query.toLowerCase().split(/\s+/).filter(Boolean)
    .every((word) => haystack.includes(word))
}
```

So "chicken noodles" finds only Chicken Pad Thai, and "feta" finds the Greek Salad through its ingredients. The query and category are also stored in the URL (for example `/?q=chicken&category=Main%20Course`), so a search can be shared and is kept when returning from a recipe. When nothing matches, the app shows a message with a "Clear search" button.

4. Project structure

```
recipe-app/
├── src/
│   ├── App.jsx                routes and recipe state (sample + user-added)
│   ├── App.css, index.css     styles and theme
│   ├── data/recipes.js        sample recipe data
│   └── components/
│       ├── Header.jsx         top navigation
│       ├── RecipeList.jsx     list page with search and filter
```

```
│       ├── SearchBar.jsx       search box and category select
│       ├── RecipeCard.jsx      card in the grid
│       ├── RecipeDetail.jsx    recipe detail component
│       ├── AddRecipeForm.jsx   form to add recipes
│       ├── NotFound.jsx        missing recipe / unknown page
│       └── ScrollToTop.jsx     resets scroll on page change
├── e2e/                        Playwright end-to-end tests
└── scripts/capture-screenshots.mjs
```

5. Testing

The app is tested with 42 automated Playwright end-to-end tests that drive a real Google Chrome browser, at both desktop size and a mobile size (Pixel 7). **All 42 tests pass.** Lint (oxlint) and the production build also pass with no errors.

Test file	Tests	What is covered
`recipe-list.spec.js`	4	All 12 recipes and card details shown; clicking a card opens the recipe; navigation links
`search.spec.js`	13	Search by title, ingredient, cuisine and tag; capitals ignored; multi-word queries; category filter alone and with text; empty results and clear; search from the URL; search kept after going back; fast typing
`recipe-detail.spec.js`	6	All fields correct; all 12 recipes open; tag links search; back button; not-found pages
`add-recipe.spec.js`	9	Validation messages; blank and invalid input rejected; errors clear while typing; saved recipe correct, listed, searchable and kept after reload; unique ids; cancel; corrupted saved data; page opens at the top
`responsive.spec.js`	5 × 2	No sideways scrolling, header on one line, search and form usable, no console errors — on desktop and mobile

To confirm the tests really detect bugs, the multi-word search was deliberately broken; 3 search tests failed as expected, and passed again once the code was restored.

Bugs found and fixed during testing

Bug	Cause	Fix
Typing quickly in the search box dropped letters	The input value was read back from the URL, which updates a moment later	Keep the text in component state and copy it to the URL afterwards
A newly saved recipe opened scrolled halfway down	The scroll position was kept when changing pages	Added `ScrollToTop`, which scrolls up on every page change
Error messages stayed after the field was fixed	Errors were only re-checked on submit	Remove a field's error as soon as it is edited
Header wrapped onto two lines on phones	Logo and links were too wide for small screens	Smaller header text and spacing below 480px

Each fix has its own test, so the bug can't come back unnoticed.

6. Use of generative AI

This project was developed with AI-assisted code generation. An AI coding assistant (Claude Code) was used to scaffold the project, generate the sample recipe data, write the React components and styles, write the end-to-end tests, and find and fix the bugs listed above. Every generated piece was reviewed and verified by running the app and the test suite.

7. How to run

```
cd recipe-app
npm install
npm run dev        # start the app, then open the printed localhost URL
npm run build      # production build
npm run test:e2e   # run the 42 end-to-end tests
```

8. Result

The result is a simple, user-friendly recipe app that meets every requirement of the project. The table below maps each rubric criterion to where it is implemented.

Rubric criterion	Implementation	Status
Project setup	Created with `npx create-vite`; React Router added	Complete
UI generation	`Header`, layout and styles	Complete
Sample data generation	12 generated recipes in `data/recipes.js`	Complete
Recipe display	`RecipeList` + `RecipeCard`	Complete
Recipe detail component	`RecipeDetail`	Complete
Search functionality	`SearchBar` + filtering in `RecipeList`	Complete
Add more recipes (task 5)	`AddRecipeForm` + localStorage	Complete

Screenshots of every feature are provided in the separate Screenshots file, and the full source code is in the Source Code archive.